231501140
Sai Senthil M

Lab Exercise 5

Cloud Object Storage Manager Using Python & MinIO

Aim

To develop a Python-based application that interacts with a local MinIO server (an S3-compatible cloud storage emulator) to understand object storage concepts such as buckets, unique object keys, and metadata tracking.

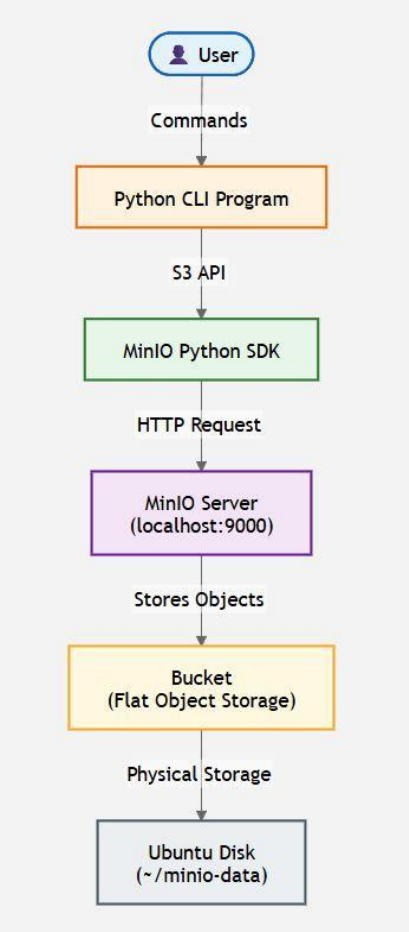
Problem Statement

Develop a Python application that mimics cloud object storage behavior using an open-source MinIO server running locally inside a VirtualBox VM, communicating via the official MinIO Python API. The application must implement:

- Create Bucket – initialize a flat container on the MinIO server.
- Upload Object – upload a local file into a bucket with a unique key.
- Download Object – retrieve an object using its unique key.
- List Bucket Contents – display objects with metadata (size, last modified).
- Delete Object – remove an object using its unique key.

Architecture

231501140
Sai Senthil M



Procedure

Step 1-2: Update Ubuntu & Install Python

```
sudo apt upgrade -y
sudo apt install python3 python3-pip -y
python3 --version
pip3 --version
```

Step 3: Install MinIO Server

```
mkdir ~/minio && cd ~/minio
wget https://dl.min.io/server/minio/release/linux-amd64/minio
chmod +x minio
sudo mv minio /usr/local/bin/
minio --version
```

Step 4-5: Create Storage Folder & Start Server

```
mkdir ~/minio-data
export MINIO_ROOT_USER=minioadmin
export MINIO_ROOT_PASSWORD=minioadmin
minio server ~/minio-data --console-address ":9001"
```

API: http://127.0.0.1:9000 | Console: http://127.0.0.1:9001 | Login: minioadmin / minioadmin

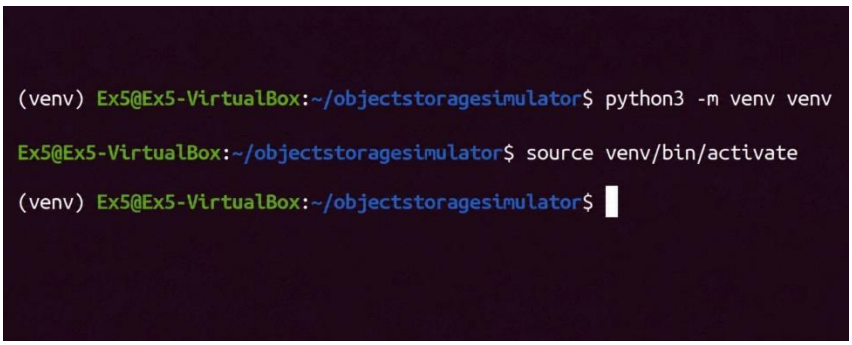
Step 6: Open MinIO Console

231501140
Sai Senthil M

Visit http://localhost:9001 and log in. Initially there are no buckets.

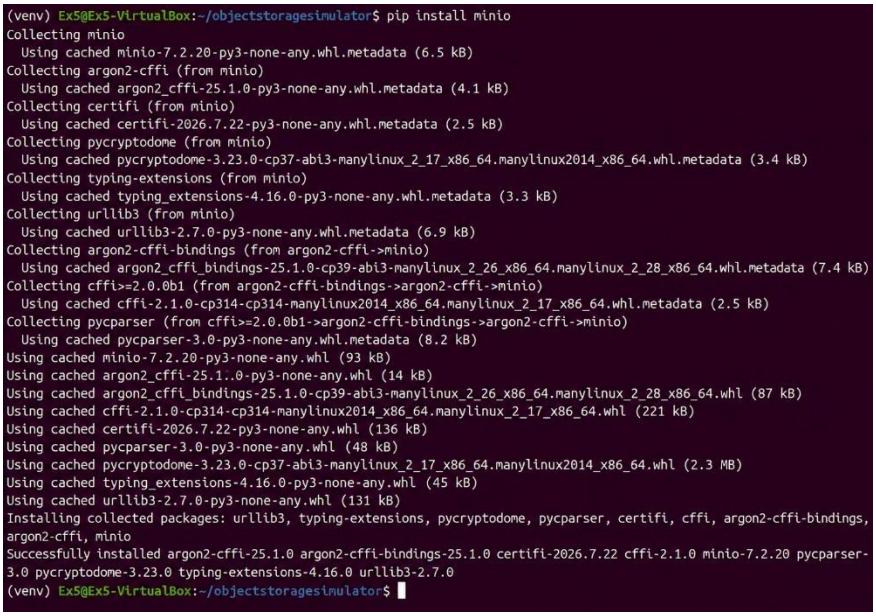
Step 7-8: Create Project & Virtual Environment

```
mkdir ObjectStorageSimulator && cd ObjectStorageSimulator
python3 -m venv venv
source venv/bin/activate
```



Step 9: Install MinIO Python SDK

```
pip install minio
```



Verify installation:

```
pip show minio
```

```
(venv) Ex5@Ex5-VirtualBox:~/objectstoragesimulator$ pip show minio
Name: minio
Version: 7.2.20
Summary: MinIO Python SDK for Amazon S3 Compatible Cloud Storage
Home-page: https://github.com/minio/minio-py
Author: MinIO, Inc.
Author-email: dev@min.io
License: Apache-2.0
Location: /home/Ex5/objectstoragesimulator/venv/lib/python3.14/site-packages
Requires: argon2-cffi, certifi, pycryptodome, typing-extensions, urllib3
Required-by:
(venv) Ex5@Ex5-VirtualBox:~/objectstoragesimulator$
```

Step 10: Create Sample File & Bucket

```
echo "Hello this is sample minio project" > sample.txt
```

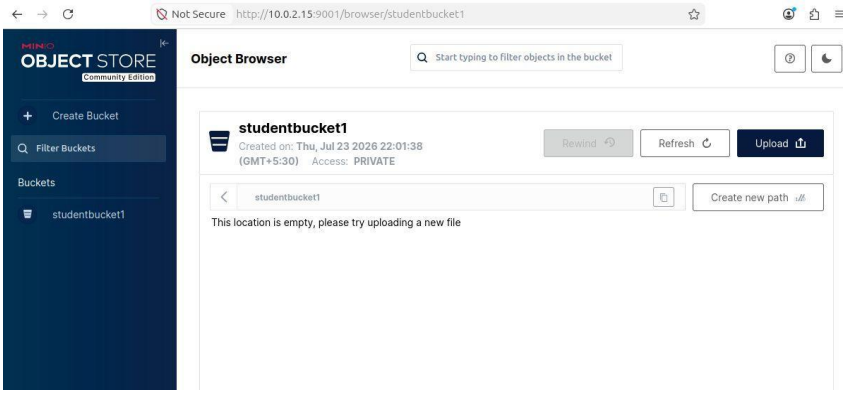
Create main.py with the bucket-creation code below and run it:

```
from minio import Minio
client = Minio("localhost:9000", access_key="minioadmin",
    secret_key="minioadmin", secure=False)
bucket_name = "studentbucket"
if not client.bucket_exists(bucket_name):
    client.make_bucket(bucket_name)
    print("Bucket Created")
else:
    print("Bucket Already Exists")
```

```
(venv) Ex5@Ex5-VirtualBox:~/objectstoragesimulator$ nano main.py
(venv) Ex5@Ex5-VirtualBox:~/objectstoragesimulator$ python main.py
Bucket Created
(venv) Ex5@Ex5-VirtualBox:~/objectstoragesimulator$
```

Verify in the MinIO console — the new bucket appears with no objects yet:

231501140
Sai Senthil M



Step 11: Upload an Object

```
client.fput_object("studentbucket", "notes.txt", "sample.txt")
print("Upload Successful")
```

Step 12: Download an Object

Modify main.py to replace the upload code with:

Replace code.

Python

```python
from minio import Minio

client = Minio(
    "localhost:9000",
    access_key="minioadmin",
    secret_key="minioadmin",
    secure=False
)

client.fget_object(
    "studentbucket1",
    "notes.txt",
    "downloaded.txt"
)

print("Download Successful")
```

Step 13: List Bucket Contents

Modify main.py to replace the previous code with the code below, which lists all objects present in the bucket along with their size and last-modified timestamp:

```
objects = client.list_objects("studentbucket")
for obj in objects:
print(obj.object_name, obj.size, obj.last_modified)
```

Running this script printed the object key, size in bytes, and last-modified timestamp for notes.txt, confirming that metadata tracking works as expected.

Step 14: Delete Object

Modify main.py to replace the previous code with the code below, which removes the uploaded object from the bucket using its unique key:

```
client.remove_object("studentbucket", "notes.txt")
print("Object Deleted")
```

Verifying in the MinIO console confirmed that notes.txt no longer appeared inside studentbucket, showing that the delete operation completed successfully.

Result

Thus, a Python application was developed to perform bucket creation, object upload, object listing, object download, and object deletion using the MinIO S3-compatible local object storage server.